Scalability

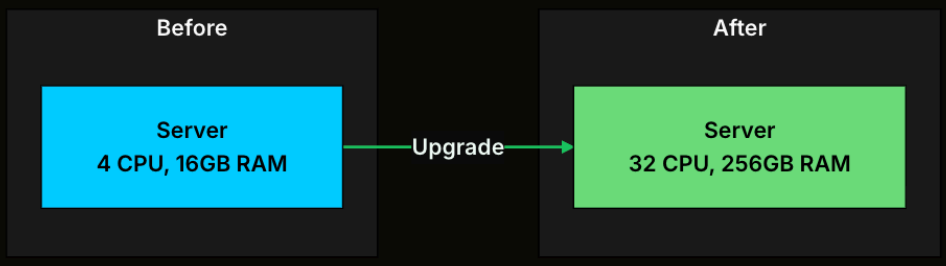
Scalability is typically evaluated along these dimensions:

Load Metrics

Metric	Description	Example
Requests per second (RPS)	Number of API calls the system handles	10,000 RPS
Concurrent users	Users active at the same time	50,000 concurrent
Data volume	Amount of data stored or processed	10 TB storage
Throughput	Data transferred per unit time	1 GB/s
Query rate	Database queries per second	50,000 QPS
Message rate	Messages processed through queues	100,000 msg/s

Vertical Scaling (Scale Up)

Vertical scaling means adding more power to your existing machines. Instead of adding more servers, you upgrade to bigger ones.



This is often the first response to performance problems because it requires no architectural changes.

Common Vertical Scaling Actions

- **Add more CPU cores** for compute-intensive workloads
- **Increase RAM** to cache more data in memory
- **Use faster SSDs** to reduce I/O bottlenecks
- **Upgrade network cards** for higher bandwidth

Pros and Cons

Pros

- **Simple:** No code changes required. Just move to a bigger machine.
- **Lower latency:** All data is local, no network hops.
- **No distributed complexity:** A single server means no network partitions, no data synchronization issues.

Cons

- **Hardware limits:** You cannot scale beyond the largest available machine. Even cloud providers have limits.
- **Single point of failure:** One server means one failure point. If it goes down, everything goes down.
- **Cost curve:** Larger machines cost disproportionately more. Doubling capacity often more than doubles cost.
- **Downtime during upgrades:** Migrating to a bigger machine typically requires downtime.

When to Use Vertical Scaling

Vertical scaling works well for:

- **Databases** where data locality matters (before sharding becomes necessary)
- **Applications with strong consistency requirements**
- **Early-stage startups** that need simplicity over scale
- **Workloads with predictable, moderate growth**

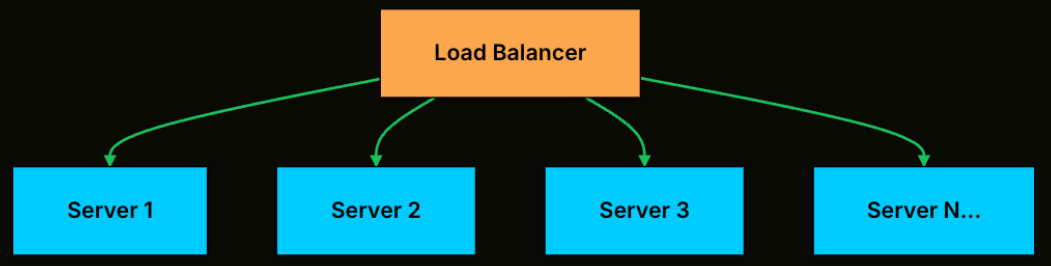
> NOTE
>
> Never dismiss vertical scaling as "not scalable." Many real-world systems run on vertically scaled databases for years. The key is knowing when horizontal scaling becomes necessary.

Horizontal Scaling (Scale Out)

Vertical scaling eventually hits a ceiling. When the biggest available machine is not big enough, or when you need fault tolerance that a single machine cannot provide, you need a different approach.

Horizontal scaling means adding more machines rather than upgrading existing ones. Instead of one powerful server, you distribute the load across many commodity servers.

This is how companies like Google, Netflix, and Amazon handle billions of requests.



Pros and Cons

Pros

- **No hard limit:** You can keep adding servers as needed. Cloud providers make this nearly unlimited.
- **Fault tolerance:** If one server fails, others continue serving traffic. No single point of failure.
- **Cost-effective:** Many smaller machines often cost less than one giant machine.
- **Geographic distribution:** You can place servers closer to users for lower latency.

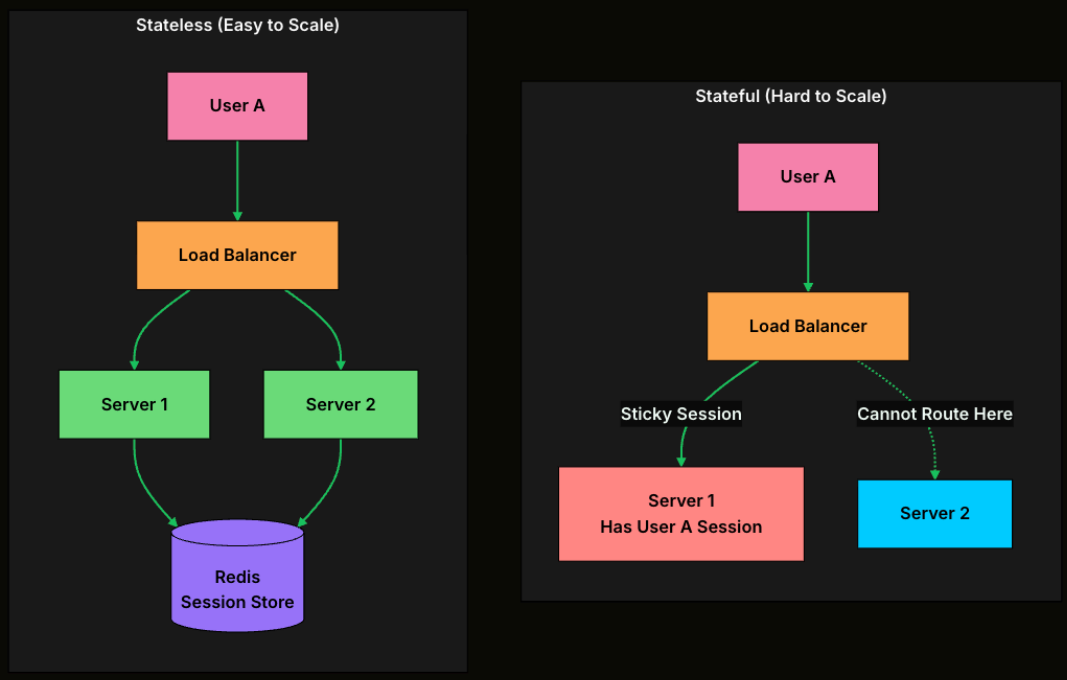
Cons

- **Complexity:** Distributed systems are harder to build, debug, and maintain.
- **Data consistency:** Keeping data synchronized across servers is challenging.
- **Network overhead:** Communication between servers adds latency.
- **Stateless requirement:** Application servers typically need to be stateless, which may require architectural changes.

Stateless vs Stateful Services

For horizontal scaling to work effectively, services should be **stateless**. A stateless service does not store any session data locally. Each request can be handled by any server.

The difference is significant for scaling:



In the stateful model, once a user's session is stored on Server 1, all their requests must go to that same server. This creates hotspots and makes it risky to remove servers. In the stateless model, session data lives in a shared store like Redis, so any server can handle any request. The load balancer has complete freedom to distribute traffic.

To make services stateless:

- Store session data in a shared cache (Redis, Memcached)
- Use tokens (JWT) instead of server-side sessions
- Store uploaded files in object storage (S3) instead of local disk

- **The Stateful Way (Server-Side Sessions):** When a user logs in, the server creates a file or a record in its memory containing the user's details. It then sends a short, random string (a Session ID) back to the user's browser. When

the user makes their next request, they send the Session ID. The server has to look up that ID in its memory to figure out who the user is. If the load balancer sends the user to a *different* server, that new server won't have the Session ID in its memory, and the user gets logged out.

- **The Stateless Way (JWT):** A JSON Web Token (JWT) solves this by packing the user's actual data (like their user ID, roles, and expiration time) directly into the token itself. When the user logs in, the server generates this JSON object, signs it cryptographically using a secret key, and sends it to the user.

Why this makes the system stateless: When the user makes their next request, they send the entire JWT. Because the token contains all the necessary information and is cryptographically signed, *any* server can verify the signature and read the user's identity instantly. The server doesn't need to remember anything from the previous request or look up a Session ID.

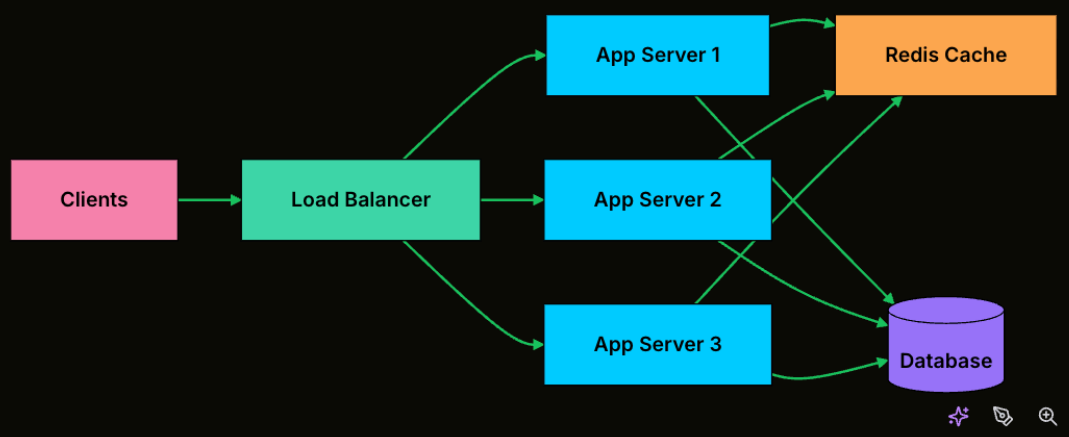
- **The Stateful Way (Local Disk):** Imagine a user uploads a new profile picture. If the application code saves that image directly to the server's local hard drive (e.g., `/var/www/uploads/profile.jpg`), that specific server is now unique. It holds state. If the load balancer routes the user's next request to Server B to view their profile, Server B will look at its own local disk, fail to find the image, and return a broken link. Furthermore, if Server A crashes and is replaced, that profile picture is gone forever.
- **The Stateless Way (Object Storage):** Services like Amazon S3, Google Cloud Storage, or Azure Blob Storage are massively scalable, standalone storage systems accessible via APIs. When a user uploads a file, your application server immediately forwards that file to S3. S3 returns a permanent URL for that file. The server then saves *only the URL* in your primary database, not the file itself.

Scaling Different Components

A typical system is not monolithic. It has multiple components, each with different scaling characteristics and challenges. Understanding these differences is crucial because the scaling strategy that works for one tier often does not work for another.

Application Tier

Application servers are usually the easiest to scale horizontally, provided they are stateless:



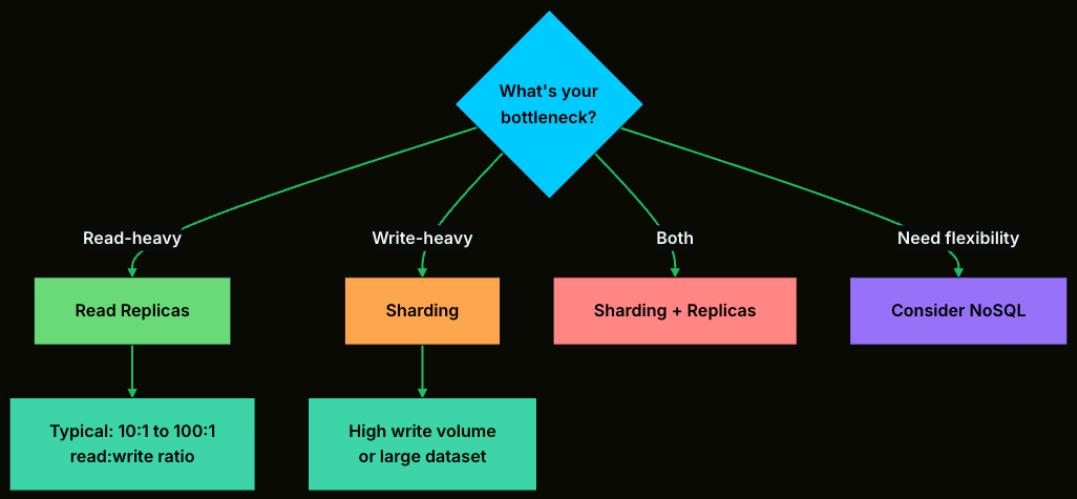
Key strategies:

- Make services stateless
- Use a load balancer to distribute traffic
- Auto-scale based on CPU, memory, or request count
- Deploy across multiple availability zones

Database Tier

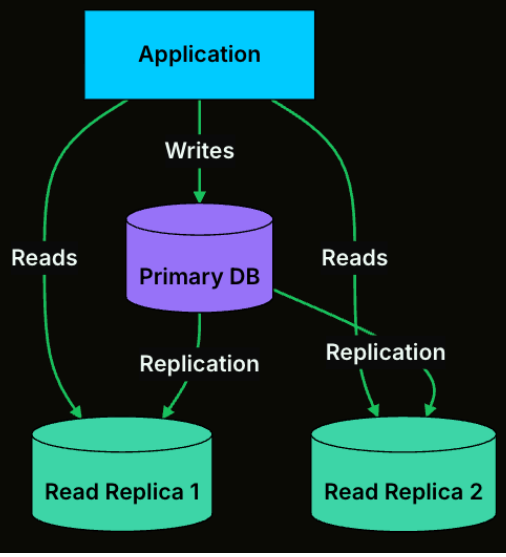
Databases are typically the hardest to scale because they manage state. Unlike application servers, you cannot simply spin up more database instances and put a load balancer in front of them. Data consistency, durability, and transaction isolation all complicate matters.

The approach depends on your workload pattern:



1. Read Replicas

For read-heavy workloads (which most applications are), create copies of your database that handle read queries:



When to use: Read-to-write ratio is 10:1 or higher, and writes are not the bottleneck.

Pros

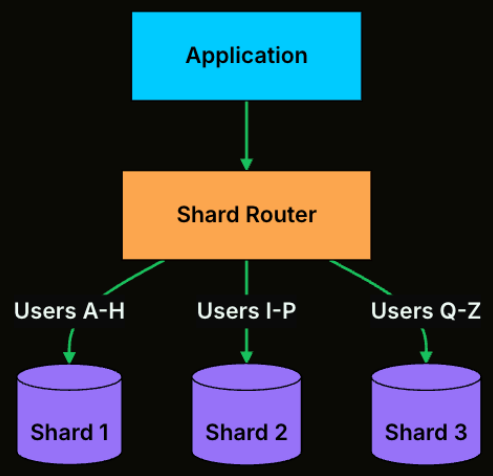
- Simple to set up (managed services handle it)
- Offloads read traffic from primary
- Provides read availability if primary fails
- No application changes for basic setup

Cons

- Does not help with write-heavy workloads
- Introduces replication lag (stale reads)
- Replicas consume storage (full data copy)
- Failover can cause brief inconsistency

2. Sharding (Partitioning)

When read replicas are not enough, or when write volume exceeds what a single primary can handle, you need to split your data across multiple databases based on a partition key:



Pros

- Distributes both reads AND writes
- Scales horizontally (add more shards)
- Each shard is smaller, faster
- Can place shards in different regions

Cons

- Complex to implement correctly
- Cross-shard queries are expensive or impossible
- Rebalancing shards is operationally difficult
- Transactions across shards are very hard

Common sharding strategies:

- **Range-based:** Shard by value ranges (A-H, I-P, Q-Z)
- **Hash-based:** Hash the key and mod by number of shards
- **Directory-based:** Maintain a lookup table mapping keys to shards

Common Sharding Strategies

1. **Range-Based Sharding:** You divide data based on ranges of a value.

◦ *Example:* Users with IDs 1 to 1,000,000 go to Shard 1. IDs 1,000,001 to 2,000,000 go to Shard 2.

◦ *The Problem:* This often creates **hotspots**. If newer users are the most active on the platform, Shard 2 will be hammered with traffic while Shard 1 sits mostly idle.

2. **Hash-Based Sharding:** You run the shard key through a mathematical hash function (e.g., `hash(user_id) % 4_shards`).
◦ *Example:* The math guarantees that `user_id: 8943` always resolves to Shard 3.
◦ *The Benefit:* This distributes data and traffic very evenly across all servers, preventing hotspots.

3. **Directory/Geo-Based Sharding:** You maintain a dedicated lookup table that acts as a map, or you shard based on logical boundaries.

◦ *Example:* All users in Asia are routed to a database cluster in Mumbai; all users in Europe are routed to Frankfurt. This is excellent for reducing latency and complying with regional data laws.

The Dark Side of Sharding

Sharding is heavily considered a "last resort" in system design because it introduces immense complexity:

• **No More Simple Joins:** If you need to write a SQL `JOIN` query that combines data from User A (on Shard 1) and User B (on Shard 2), the database can no longer do it locally. The application has to pull data from both servers over the network and stitch it together in memory.

• **Resharding:** If your 4 shards fill up and you need to add a 5th, your hashing math changes (`hash(user_id) % 5`). Suddenly, millions of users are pointing to the wrong servers, and you have to migrate massive amounts of data without taking the system offline.

3. NoSQL Databases

NoSQL databases like Cassandra, MongoDB, and DynamoDB are designed for horizontal scaling from the ground up:

- **Built-in sharding:** Data is automatically distributed
- **Eventual consistency:** Trade strong consistency for availability
- **No joins:** Data model must accommodate denormalization

Pros

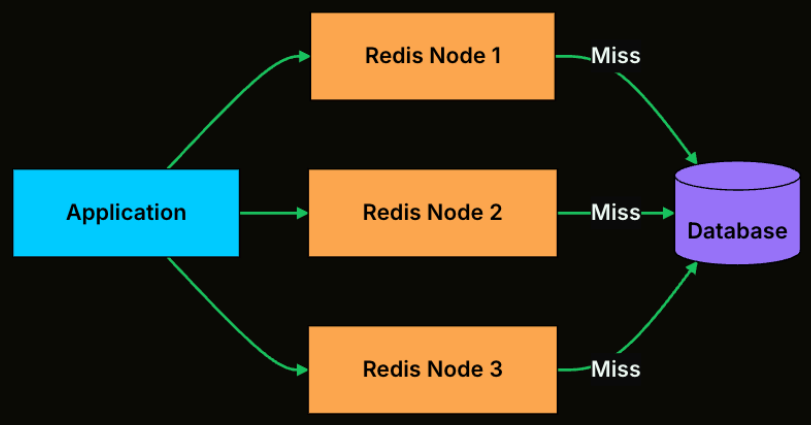
- Built-in sharding (automatic distribution)
- Designed for horizontal scale
- Often better write performance
- Schema flexibility

Cons

- Different query patterns than SQL
- No joins (denormalization required)
- Eventual consistency in many cases
- Less tooling ecosystem than SQL

Caching Tier

Caching reduces load on databases and improves response times. A well-designed cache can handle 100x the throughput of a database, making it essential for high-traffic systems. Redis, for example, can handle 100,000+ operations per second on a single node.



Cache Scaling strategies:

- **Redis Cluster:** Automatically partitions data across nodes using hash slots
- **Consistent hashing:** Distributes keys evenly and minimizes redistribution when nodes are added or removed
- **Cache-aside pattern:** Application checks cache first, falls back to database on cache miss, then populates the cache

1. The Cache-Aside Pattern (The Green Arrows)

1. Check Cache: The application needs a user's profile. It asks Redis Node 1 for the data.
2. Cache Hit / Miss: If Redis has it (a "Hit"), it returns it immediately. If not, you get the "Miss" shown in the diagram.
3. Fallback to DB: The application queries the primary Database to get the profile.
4. Populate Cache: The application takes that database response and writes it into Redis. The next time anyone asks for that profile, it will be a Cache Hit.

2. The Scaling Problem (Standard Hashing)

When you have 3 Redis Nodes , the load balancer needs to know which node

holds which cached item. The naive approach is a standard modulo hash: `hash(key) % 3`.
If `user_123` hashes to the number 10, then $10 mod 3 = 1$. The data goes to Node 1.
The Catastrophe: What happens when Black Friday hits, traffic spikes, and you need to add a 4th Redis Node?
Your math changes to `hash(key) % 4`. Now, $10 mod 4 = 2$. The load balancer looks for `user_123` on Node 2, gets a Cache Miss, and hammers the database. In fact, adding a single node changes the math for nearly *every* key, causing a massive wave of cache misses that can instantly crash your database.

3. The Solution: Consistent Hashing

Instead of a modulo array, imagine a logical ring of numbers from 0 to 359 (like degrees on a compass).

1. Place the Servers

You have three physical computers (Redis Node 1, Node 2, and Node 3) sitting in a data center. We need to assign them fixed spots on our painted circle.

- We feed the name **"Node 1"** into our magic machine. The machine spits out **90**. We walk over to the 90-degree mark on the circle and place a bucket there.
- We feed **"Node 2"** into the machine. It spits out **210**. We put a bucket at 210.
- We feed **"Node 3"** into the machine. It spits out **300**. We put a bucket at 300.

2. Place the Keys (The Data)

Now, hundreds of users are asking for different discount coupons. We need to figure out which bucket (server) should hold each coupon.

- A user asks for **"Coupon_A"**. We feed "Coupon_A" into our magic machine. The machine spits out **45**. We place "Coupon_A" at the 45-degree mark on the circle.

- A user asks for **"Coupon_B"**. We feed it into the machine. It spits out **250**. We place it at the 250-degree mark.

The Clockwise Rule (How they connect)

The rule is simple: Start at the coupon's location and walk clockwise around the circle until you bump into a bucket.

- **"Coupon_A"** is at 45. If you walk clockwise, the very first bucket you hit is **Node 1** at 90. Therefore, Node 1 is responsible for storing Coupon A.
- **"Coupon_B"** is at 250. If you walk clockwise, the first bucket you hit is **Node 3** at 300. Node 3 stores Coupon B.

Here is the difference in the "blast radius" between the two methods:

The Old Way (Standard Modulo Hashing)

If you have 300,000 cached coupons and 3 servers, you use `hash(key) % 3` to place them.
If you add a 4th server, the formula changes to `hash(key) % 4` for **every single key in the entire system**.
Because the denominator changed, the math output changes for almost everything. Suddenly, roughly 75% of your keys (225,000 coupons) are pointing to the wrong servers. **Result: 225,000 simultaneous cache misses.** Your database crashes instantly.

The New Way (Consistent Hashing)

Imagine those same 300,000 coupons scattered around the ring, with 100,000 coupons going to each of the 3 servers.

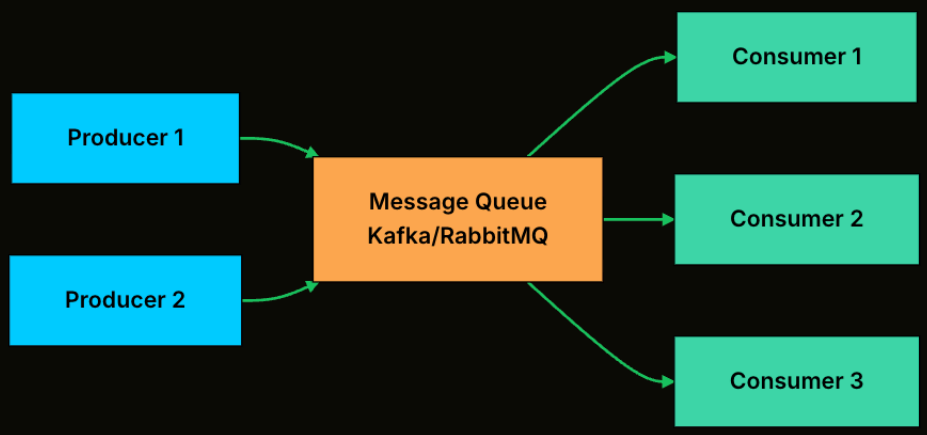
You add Store 4 to the ring.
Store 4 lands in *one specific spot* (let's say, right between Store 1 and Store 3).

- Store 4 intercepts keys `a, b, c` (and let's say about 25,000 other keys that were previously walking toward Store 1).
- **Those 25,000 keys get a cache miss.** They hit the database.

To prevent even that 25% spike from hitting the database, engineers use a technique called **Cache Warming**. Before they allow the load balancer to send live user traffic to the newly added Store 4, they have a background script secretly pull `a, b, c` from the database and pre-load them into Store 4.

Message Queue Tier

Message queues are essential for scaling asynchronous workloads. They decouple producers from consumers, allowing each to scale independently, and they buffer traffic spikes so consumers can process at their own pace.



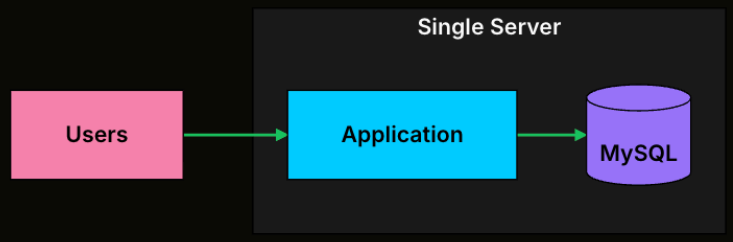
How queues help scalability:

- **Decouple producers and consumers:** Scale each independently
- **Buffer traffic spikes:** Queue absorbs bursts, consumers process at their own pace
- **Partition topics:** Kafka partitions allow parallel consumption

Example: Scaling from 0 to millions of users

Theory is useful, but seeing scalability in action makes it stick. Let us walk through how a startup might scale a social media application from zero to millions of users. Each stage solves a specific bottleneck that emerged from growth.

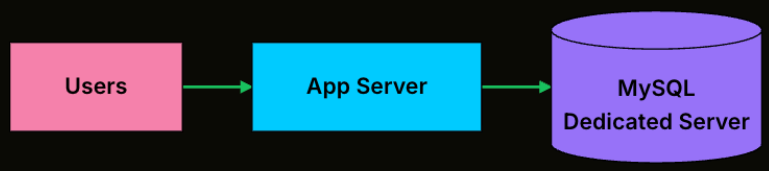
Stage 1: Single Server (0-10K users)



At launch, everything runs on one machine. The application and database share the same server. This setup is simple, cheap, and perfectly adequate for a few thousand users. There is no distributed system complexity, no network latency between components, and debugging is straightforward.

The bottleneck emerges when the application and database start competing for CPU and memory on the same machine.

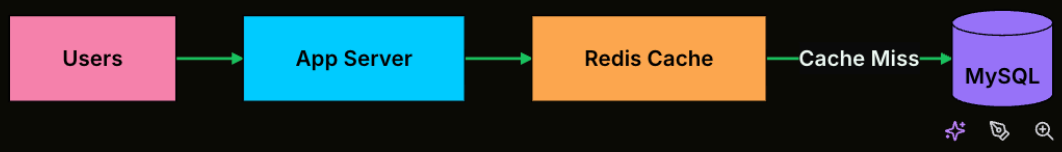
Stage 2: Separate Database (10K-100K users)



The first scaling move is usually separating the database onto its own machine. Now each component can be tuned independently. You can give the database server more RAM for caching, while the app server gets more CPU for request processing.

The bottleneck shifts to the database. As user counts grow, the database handles more queries, and read operations start slowing down.

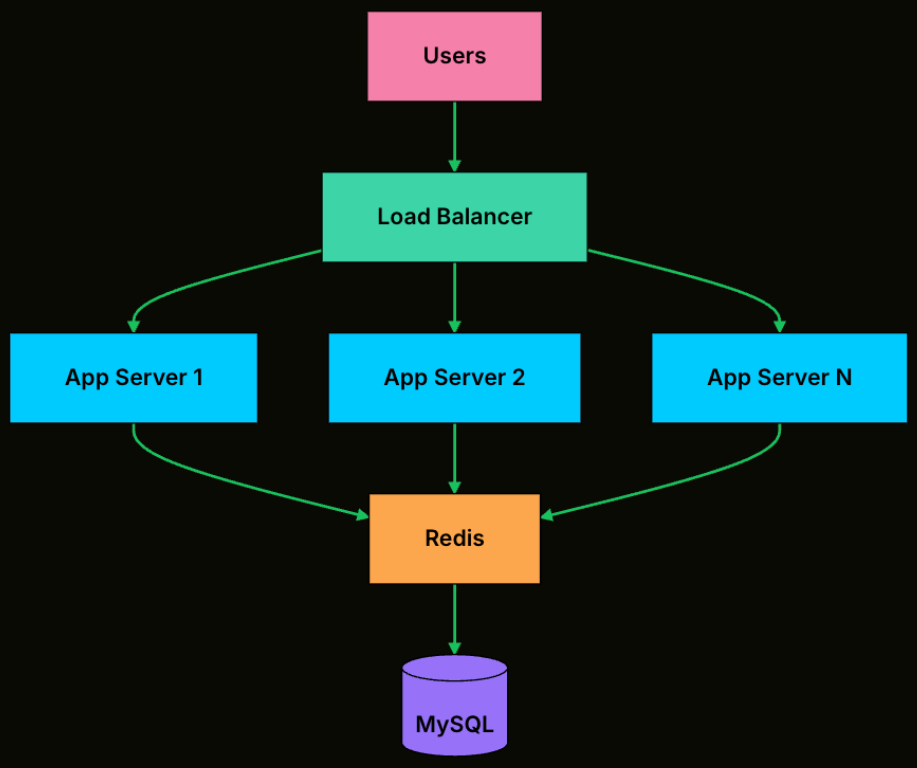
Stage 3: Add Caching (100K-500K users)



Adding a cache layer dramatically reduces database load. Hot data, things like user profiles, recent posts, and session data, gets served from memory. Redis can handle hundreds of thousands of reads per second, far more than MySQL. With a good caching strategy, 80-90% of reads never hit the database.

The bottleneck is now the single app server. It cannot handle the incoming request volume.

Stage 4: Multiple App Servers (500K-2M users)

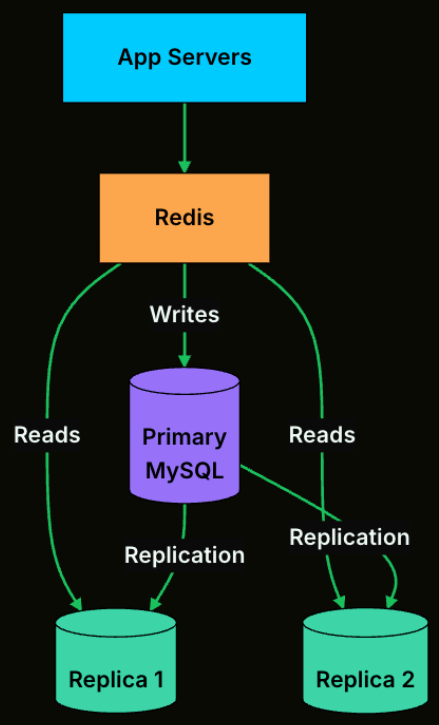


This is where horizontal scaling begins. A load balancer distributes traffic across multiple app servers. Each server is stateless, storing no session data locally. The Redis cache serves as the shared session store.

Adding more app servers is now trivial. Need more capacity? Spin up another server. Traffic spike during peak hours? Auto-scaling adds servers automatically.

The bottleneck shifts back to the database. With more app servers generating more queries, the single MySQL instance becomes overwhelmed.

Stage 5: Read Replicas (2M-10M users)

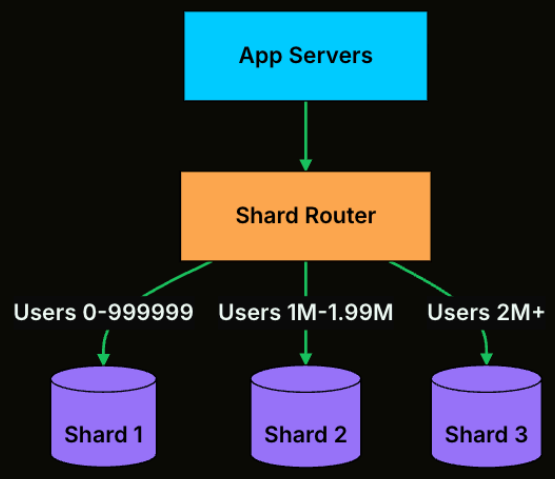


Most applications are read-heavy, with reads outnumbering writes by 10:1 or more. Read replicas take advantage of this pattern. The primary database handles all writes, while replicas serve read queries. This multiplies read capacity without changing the application much.

The trade-off is replication lag. Replicas may be a few milliseconds behind the primary, so recently written data might not be immediately visible on reads. For most applications, this is acceptable.

The bottleneck becomes write throughput. One primary database can only handle so many writes per second.

Stage 6: Sharding (10M+ users)



Sharding is the final frontier of relational database scaling. Data is partitioned across multiple databases based on a shard key, typically user ID. Each shard handles a subset of users, distributing both read and write load.

This is powerful but comes with significant complexity. Cross-shard queries become expensive or impossible. Rebalancing shards when they grow unevenly is operationally challenging. Many teams at this stage consider moving to distributed databases like CockroachDB or Vitess that handle sharding automatically.

Summary

Scalability is about designing systems that can grow with demand without falling apart. Here are the key takeaways:

- **Vertical scaling** is simple but has limits. Use it for databases, early-stage systems, and workloads where simplicity matters more than infinite growth.
- **Horizontal scaling** is more complex but can grow indefinitely. It requires stateless services and careful data management.
- **Different components scale differently.** App servers are easy to scale horizontally. Databases are hard because they manage state. Know the difference.
- **Always identify the bottleneck first** before deciding how to scale. Adding more app servers does not help if the database is the problem.
- **Common patterns** like load balancing, caching, async processing, and database optimization appear in almost every scalable system.

Scalability addresses the question of handling more load. But a system that can handle a million users is not very useful if it goes down every week. A scaled system that crashes still fails your users.

This brings us to our next topic: **availability**. How do you ensure your system stays operational even when individual components fail?